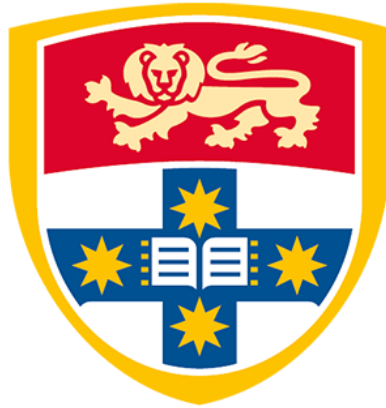




THE UNIVERSITY OF
SYDNEY

AERO3261
Assignment 1

PROPELLER PERFORMANCE AND ENGINE/AIRFRAME MATCHING

Harry Lieshout : 530619370

Abstract

This report investigates the performance of an APC20x8 propeller using Blade Element Momentum (BEM) theory and its integration with a given engine and airframe. Aerodynamic data for the NACA4412 airfoil were obtained using XFOIL and incorporated into a MATLAB-based BEM model to evaluate thrust, power, and efficiency across a range of operating conditions. The propeller was then matched with the engine to determine operating points and assess aircraft performance in terms of maximum speed and rate of climb. The effects of propeller pitch variation and engine power scaling were analysed, highlighting key trade-offs between cruise and climb performance. The results demonstrate that aircraft performance is governed by the interaction between aerodynamic loading, advance ratio, and available power, providing insight into optimal propulsion system selection.

Contents

1	Introduction	1
1.1	Background	1
2	Propeller Geometry	1
2.1	Selected Propeller - APC20x8	1
2.2	Pitch and Chord Distribution	1
3	Airfoil Aerodynamic Data (XFOIL)	2
3.1	Reynolds Numbers Considered	2
3.2	Lift and Drag Characteristics	3
3.3	Discussion of Reynolds Number Effects	3
4	Blade Element Momentum (BEM) Propeller Analysis	4
4.1	MATLAB Implementation	4
4.2	Thrust and Power Performance	4
4.3	Non-Dimensional Performance Coefficients	5
4.4	Discussion of BEM Results	5
5	Engine–Propeller–Airframe Matching	5
5.1	Aircraft Model	5
5.2	Cruise and Climb Matching	5
5.3	Flight Conditions	6
5.4	Cruise and Climb Matching	6
5.5	Specific Fuel Consumption Estimation	7
5.6	Discussion	8
6	Impact of Propeller Pitch Setting	8
6.1	Discussion	9
7	Engine scaling	9
7.1	Discussion	9
8	Overall Discussion	10

9 Conclusion	10
A Appendix	12
B Main MATLAB Code	12

List of Figures

1	Propeller geometric properties along the blade radius for the APC 20x8 propeller	2
2	Airfoil aerodynamic coefficients for different Reynolds numbers	3
3	Propeller thrust and power characteristics for multiple RPM values . . .	4
4	Non-dimensional propeller performance coefficients as a function of advance ratio J for multiple RPM values	5
5		6
6	Variation of operating parameters with flight speed at different throttle values	7
7	Effect of pitch setting on aircraft performance	8
8	Effect of engine power scaling on aircraft performance	9
9	C_t vs advance ratio J	12

List of Tables

1	Propeller Geometry Distribution (APC20x8)	1
2	Baseline Performance Summary	7
3	Cruise Operating Point Summary	7
4	Climb Operating Point Summary	7
5	Summary of pitch setting effects at 100% throttle	8
6	Summary of engine size effects at 100% throttle	9

1 Introduction

1.1 Background

Propeller-driven propulsion systems are widely used in small unmanned aerial vehicles (UAVs) due to their simplicity, efficiency, and reliability at low to moderate flight speeds. The performance of such systems is strongly dependent on the interaction between the propeller, engine, and airframe. Understanding this interaction is essential for optimising aircraft performance, particularly in terms of cruise speed and climb capability.

Blade Element Momentum (BEM) theory provides a practical framework for predicting propeller performance by combining aerodynamic blade element analysis with momentum theory. When coupled with engine characteristics and airframe drag models, it enables a complete assessment of propulsion system performance.

2 Propeller Geometry

2.1 Selected Propeller - APC20x8

Table 1: Propeller Geometry Distribution (APC20x8)

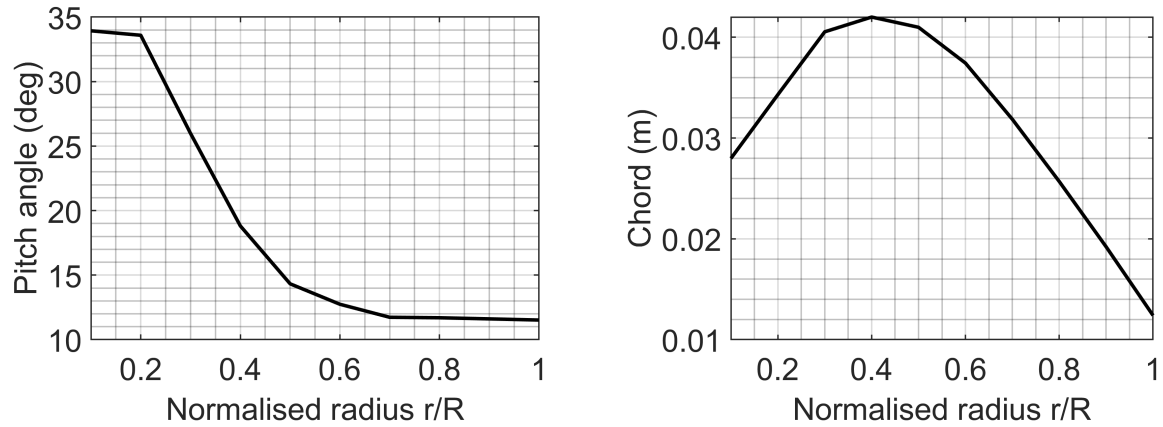
r/R	Pitch (rad)	Chord (m)
0.1	0.5923	0.0280
0.2	0.5862	0.0343
0.3	0.4534	0.0405
0.4	0.3284	0.0420
0.5	0.2500	0.0410
0.6	0.2224	0.0374
0.7	0.2047	0.0319
0.8	0.2041	0.0257
0.9	0.2026	0.0192
1.0	0.2010	0.0124

2.2 Pitch and Chord Distribution

The geometric properties of the selected APC20x8 propeller are shown in Figure 1. The pitch angle varies along the blade radius to maintain an appropriate angle of attack for the blade elements as the local tangential velocity increases with radius. Near the root, the pitch angle is higher to compensate for the lower rotational speed, while toward the tip the pitch angle decreases.

The chord distribution also varies along the blade span. The chord is generally larger near the root to provide sufficient structural strength and to generate lift where the blade speed is lower. Toward the tip the chord decreases, reducing drag and minimizing tip losses.

These geometric variations are typical for propeller blades designed using blade element momentum principles, ensuring efficient thrust production across the span of the propeller.



(a) Variation of pitch along the propeller blade for the APC 20x8 propeller (r/R) (b) Variation of chord along the propeller blade for the APC 20x8 propeller (r/R)

Figure 1: Propeller geometric properties along the blade radius for the APC 20x8 propeller

3 Airfoil Aerodynamic Data (XFOIL)

3.1 Reynolds Numbers Considered

The aerodynamic characteristics of the blade airfoil were obtained using XFOIL for three Reynolds numbers representative of the operating conditions along the propeller blade. The Reynolds numbers considered were:

- $Re = 6.5 \times 10^5$
- $Re = 8.0 \times 10^5$
- $Re = 1.0 \times 10^6$

These values correspond to the expected range of Reynolds numbers encountered by the blade elements from mid-span to near the tip during propeller operation. Generating aerodynamic data across this range allows the blade element momentum analysis to account for variations in aerodynamic performance due to changes in local flow conditions.

3.2 Lift and Drag Characteristics

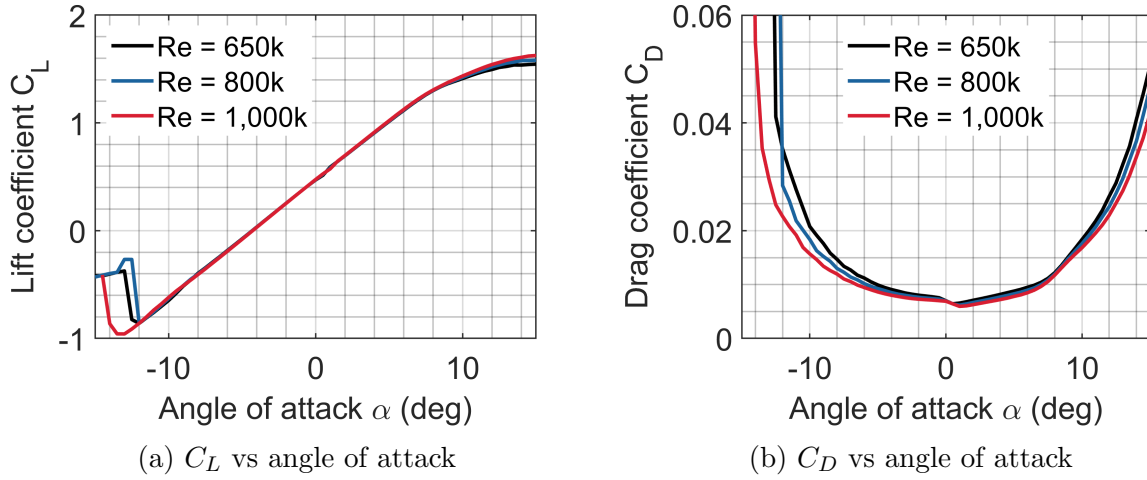


Figure 2: Airfoil aerodynamic coefficients for different Reynolds numbers

The aerodynamic performance of the NACA4412 airfoil used in the propeller blades was analysed using XFOIL. Figure 2 shows the variation of lift coefficient C_L and drag coefficient C_D with angle of attack for the three Reynolds numbers considered.

The lift coefficient increases approximately linearly with angle of attack in the pre-stall region, which is consistent with thin airfoil theory. The stall occurs at angles of attack greater than approximately 12° to 14° , where the lift curve begins to deviate from linear behaviour.

The drag coefficient remains relatively low near zero angle of attack and increases as the angle of attack increases due to the growth of viscous and pressure drag components.

3.3 Discussion of Reynolds Number Effects

The Reynolds number has a noticeable effect on the aerodynamic performance of the airfoil. As the Reynolds number increases, the lift curve becomes slightly steeper and the maximum lift coefficient increases. This occurs because higher Reynolds numbers reduce the relative influence of viscous effects, allowing the boundary layer to remain attached over a larger portion of the airfoil surface.

Similarly, the drag coefficient decreases with increasing Reynolds number, particularly at moderate angles of attack. Lower Reynolds numbers result in thicker boundary layers and earlier flow separation, which increases drag.

These trends are important for propeller analysis because different sections of the blade operate at different Reynolds numbers depending on the local velocity and chord length. Accurate aerodynamic data across this range is therefore required for reliable performance prediction using blade element momentum theory.

4 Blade Element Momentum (BEM) Propeller Analysis

Blade Element Momentum (BEM) theory combines blade element theory and momentum theory to predict propeller performance. The blade is divided into small radial elements, and the forces on each element are calculated using local flow conditions and airfoil data.

The local inflow velocity at each blade section is determined by the combination of axial and rotational components. The advance ratio is defined as:

$$J = \frac{V}{nD}$$

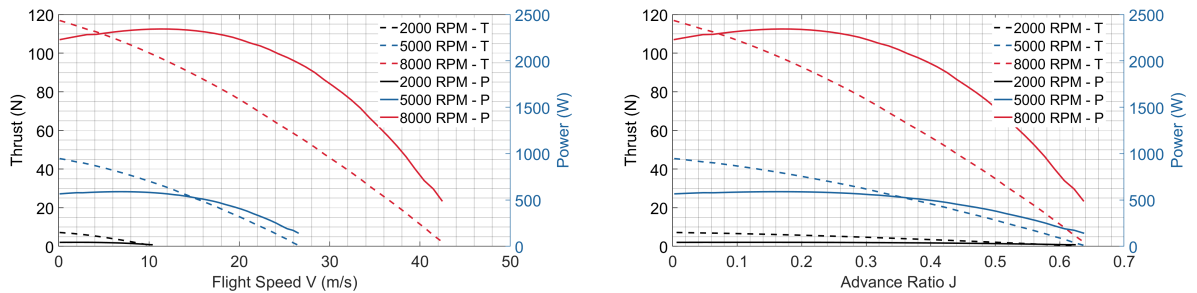
where V is the flight speed, n is the rotational speed, and D is the propeller diameter.

Using aerodynamic coefficients obtained from XFOIL, the elemental lift and drag forces are resolved into thrust and torque contributions. These are integrated along the blade span to obtain total thrust and torque.

4.1 MATLAB Implementation

The BEM model was implemented in MATLAB using the provided code structure. The propeller geometry was updated using the measured chord and pitch distributions, and aerodynamic coefficients were interpolated from XFOIL data. The analysis was performed over a range of flight speeds for multiple rotational speeds (2000, 5000, and 8000 RPM). At each operating condition, the induced velocity factors were solved iteratively until convergence was achieved. The outputs of the model include thrust, torque, power, and non-dimensional performance coefficients as functions of advance ratio and flight speed.

4.2 Thrust and Power Performance



(a) Thrust and power variation with flight speed (b) Thrust and power variation with advance ratio J

Figure 3: Propeller thrust and power characteristics for multiple RPM values

4.3 Non-Dimensional Performance Coefficients

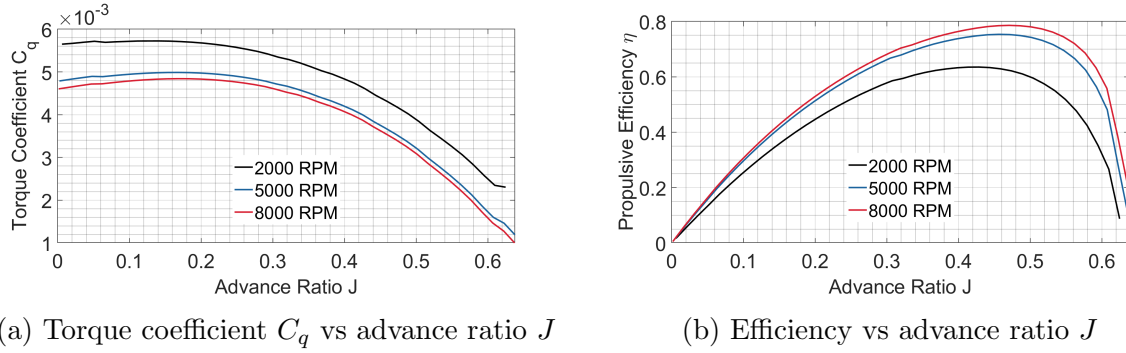


Figure 4: Non-dimensional propeller performance coefficients as a function of advance ratio J for multiple RPM values

4.4 Discussion of BEM Results

The BEM results presented in Figures ?? and 4 demonstrate clear trends in propeller performance as a function of advance ratio J . The thrust coefficient C_T decreases consistently with increasing J , indicating that as forward velocity increases, the effective angle of attack of the blade elements reduces, leading to lower thrust generation.

The torque coefficient C_Q follows a similar trend, although its reduction with J is less steep than that of C_T . This suggests that thrust is more sensitive to changes in inflow conditions than torque. At low advance ratios, the propeller operates in a heavily loaded condition, producing high thrust but with significant induced losses.

The efficiency curves show a distinct peak at an intermediate advance ratio, corresponding to the optimal operating condition. At low J , efficiency is low due to excessive induced losses, while at high J , efficiency decreases again as the propeller becomes under-loaded and generates insufficient lift. The peak efficiency occurs where the balance between thrust production and power input is maximised.

Increasing RPM increases thrust and power across all advance ratios, effectively scaling the performance curves upward. However, the location of the peak efficiency remains largely unchanged, indicating that RPM primarily affects magnitude rather than the underlying aerodynamic behaviour.

5 Engine–Propeller–Airframe Matching

5.1 Aircraft Model

5.2 Cruise and Climb Matching

The engine–propeller matching was performed by equating the torque produced by the engine with the torque required by the propeller at each flight speed. This determines the operating RPM of the system.

The aircraft performance is governed by the relation:

$$T = D + W \sin \theta$$

where T is thrust, D is drag, W is weight, and θ is the climb angle.

The maximum flight speed occurs at the point where thrust equals drag. At this condition, there is no excess thrust available for acceleration or climb.

The rate of climb is determined by the excess thrust:

$$RoC = \frac{(T - D)V}{W}$$

This indicates that climb performance depends on both the magnitude of excess thrust and the flight speed.

The results show that increasing throttle increases thrust across all speeds, leading to higher maximum speeds and improved climb performance. The optimal climb speed occurs at a lower velocity than the maximum speed, where the product of excess thrust and velocity is maximised.

5.3 Flight Conditions

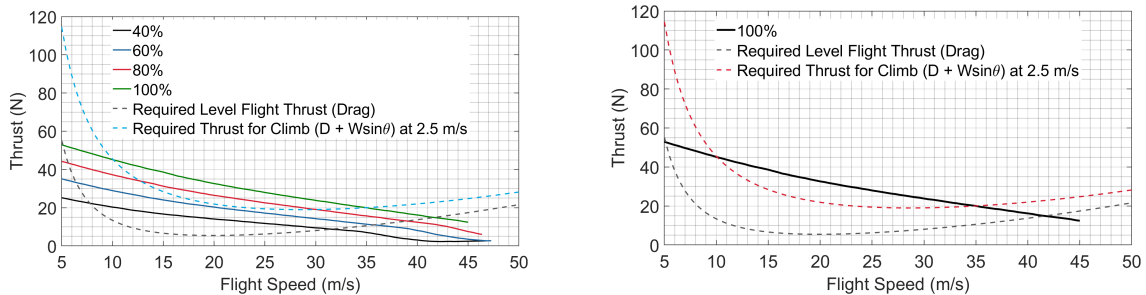
The analysis was conducted at a representative operating altitude of $h = 1680$ m, corresponding to typical mission conditions for the UAV. Atmospheric properties at this altitude were specified as a static pressure of $P = 82,750$ Pa and a temperature of $T = 277.23$ K.

Assuming ideal gas behaviour, the air density was determined using the relation:

$$\rho = \frac{P}{RT}$$

where $R = 287.05$ J/kg/K is the specific gas constant for air. These atmospheric conditions were used consistently throughout the propeller performance analysis and engine-propeller-airframe matching to ensure realistic estimation of aerodynamic forces and propulsion performance.

5.4 Cruise and Climb Matching



(a) Thrust available and airframe drag vs flight speed (b) Engine-propeller thrust matching at 100% throttle

Figure 5

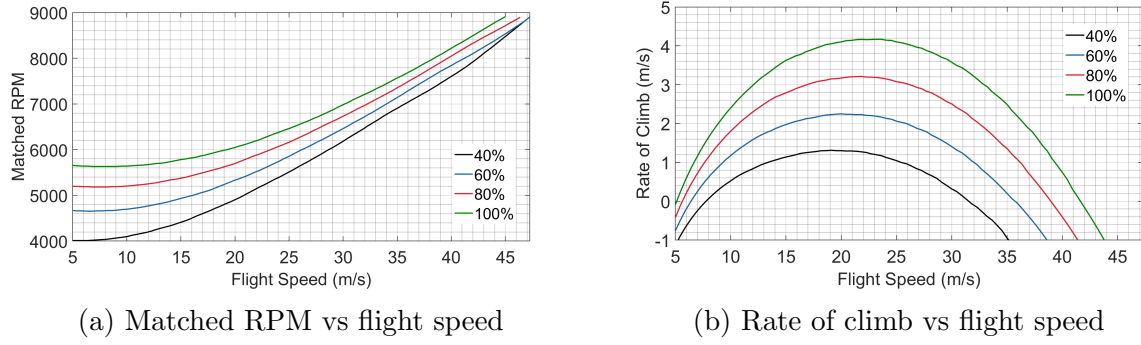


Figure 6: Variation of operating parameters with flight speed at different throttle values

Table 2: Baseline Performance Summary

Throttle (%)	$V_{\max}$ (m/s)	$\text{RPM}_{V_{\max}}$	$\text{RoC}_{\max}$ (m/s)	$V_{\text{best climb}}$ (m/s)	$\text{RPM}_{\text{RoC}_{\max}}$
40	31.36	6382	1.31	19.09	4811
60	35.46	7204	2.25	20.00	5333
80	38.64	7853	3.21	21.82	5868
100	41.36	8403	4.17	23.64	6347

Table 3: Cruise Operating Point Summary

Throttle (%)	RPM	η	$\dot{m}_f$ (kg/s)	P_{eng} (W)	P_{prop} (W)	SFC (kg/Ns)
40	6382	0.640	4.67×10^{-5}	432	433	5.28×10^{-6}
60	7204	0.637	6.19×10^{-5}	617	617	5.58×10^{-6}
80	7853	0.638	7.76×10^{-5}	799	802	5.86×10^{-6}
100	8403	0.637	9.22×10^{-5}	977	979	6.12×10^{-6}

Table 4: Climb Operating Point Summary

Throttle (%)	RPM	η	$\dot{m}_f$ (kg/s)	P_{eng} (W)	P_{prop} (W)	SFC (kg/Ns)
40	4811	0.786	3.57×10^{-5}	353	354	2.45×10^{-6}
60	5333	0.784	4.80×10^{-5}	517	519	2.36×10^{-6}
80	5868	0.783	6.22×10^{-5}	696	699	2.48×10^{-6}
100	6347	0.783	7.64×10^{-5}	880	883	2.61×10^{-6}

5.5 Specific Fuel Consumption Estimation

The specific fuel consumption (SFC) was calculated as the ratio of fuel mass flow rate to thrust produced. The results indicate that SFC increases with throttle setting, reflecting the higher fuel consumption required to generate increased power output.

Lower SFC values are observed at climb conditions compared to cruise, as the propeller operates at higher efficiency and produces greater useful thrust relative to fuel consumption. This highlights the importance of operating conditions in determining propulsion efficiency.

5.6 Discussion

The engine-propeller matching results illustrate how the operating point of the propulsion system is determined by the balance between engine torque and propeller torque requirements. For each flight speed, the propeller adjusts its operating RPM such that the torque required matches that supplied by the engine.

From the results, the maximum cruise speed for the baseline configuration is approximately 41.36 m/s, corresponding to the point where thrust equals drag. At lower speeds, thrust exceeds drag, resulting in positive excess thrust and enabling climb. The maximum rate of climb is approximately 4.17 m/s, occurring where the difference between thrust and drag is greatest.

As flight speed increases, the advance ratio increases, reducing propeller loading and therefore thrust. Simultaneously, drag increases rapidly with speed. The intersection of these two trends defines the maximum achievable speed of the aircraft.

Increasing throttle shifts the thrust curve upward, allowing higher thrust levels at all speeds. This results in both an increase in maximum speed and a significant increase in excess thrust, which directly improves climb performance. These results highlight that aircraft performance is governed by the interaction between propeller aerodynamics, engine characteristics, and airframe drag.

6 Impact of Propeller Pitch Setting

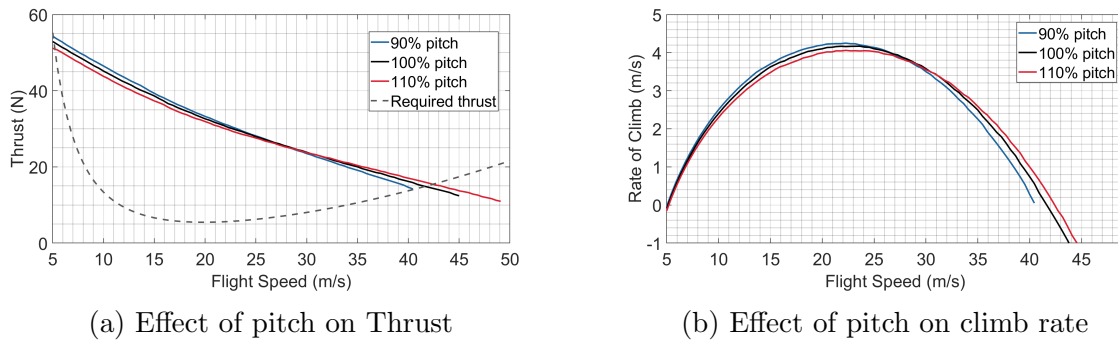


Figure 7: Effect of pitch setting on aircraft performance

Table 5: Summary of pitch setting effects at 100% throttle

Pitch scale	$V_{\max}$ (m/s)	$RoC_{\max}$ (m/s)
0.9	40.46	4.25
1.0	41.36	4.17
1.1	42.27	4.06

6.1 Discussion

The effect of pitch variation on aircraft performance is clearly demonstrated by the results in Table ???. Increasing the pitch scale from 0.9 to 1.1 results in an increase in maximum speed from 40.46 m/s to 42.27 m/s, corresponding to an increase of approximately 4.5%. This indicates that higher pitch settings improve high-speed performance by better matching the propeller to larger advance ratios. However, this improvement in cruise performance comes at the expense of climb capability. The maximum rate of climb decreases from 4.25 m/s at a pitch scale of 0.9 to 4.06 m/s at 1.1, representing a reduction of approximately 4.5%. This occurs because increasing pitch reduces the effective angle of attack at low speeds, decreasing thrust generation in the climb regime. The baseline pitch setting (scale = 1.0) provides a balanced compromise, with a maximum speed of 41.36 m/s and a climb rate of 4.17 m/s. This configuration achieves near-optimal cruise performance while maintaining only a small reduction in climb capability relative to the lower pitch setting.

7 Engine scaling

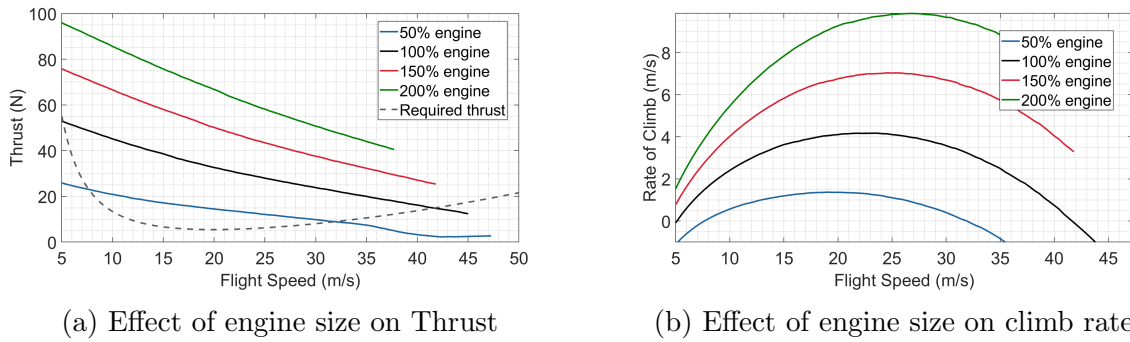


Figure 8: Effect of engine power scaling on aircraft performance

Table 6: Summary of engine size effects at 100% throttle

Engine scale factor	$V_{\max}$ (m/s)	$RoC_{\max}$ (m/s)
0.5	31.82	1.37
1.0	41.36	4.17
1.5	41.82	7.03
2.0	37.73	9.85

7.1 Discussion

The effect of engine scaling on aircraft performance is clearly reflected in the results presented in Table ???. Increasing the engine scale from 0.5 to 1.0 results in a substantial increase in maximum speed from 31.82 m/s to 41.36 m/s, indicating that the aircraft is power-limited at low engine sizes.

Further increases in engine size yield diminishing returns in maximum speed. Increasing the scale from 1.0 to 1.5 results in only a marginal increase to 41.82 m/s (approximately 1.1%), while increasing the scale to 2.0 leads to a reduction in maximum speed

to 37.73 m/s. This indicates that the aircraft transitions from a power-limited regime to a drag-limited regime, where additional power does not translate into higher cruise speed. In contrast, the rate of climb increases significantly with engine scaling. The climb rate increases from 4.17 m/s at baseline to 7.03 m/s at a scale of 1.5 and 9.85 m/s at a scale of 2.0, more than doubling overall. This is because climb performance depends on excess thrust, which continues to increase with engine power even when maximum speed is constrained by drag. These results demonstrate that increasing engine size is highly effective for improving climb performance, while gains in maximum speed are limited once aerodynamic drag becomes dominant.

8 Overall Discussion

The results of this study demonstrate that propeller performance is primarily governed by advance ratio, which dictates the aerodynamic loading of the blade elements. At low advance ratios, the propeller produces high thrust but operates inefficiently due to significant induced losses. At higher advance ratios, the propeller becomes under-loaded, resulting in reduced thrust despite improved aerodynamic efficiency. The interaction between propeller characteristics and engine performance determines the operating envelope of the aircraft. The engine torque curve defines the achievable RPM, while the propeller converts this into thrust, which must overcome airframe drag to enable flight. Pitch variation provides a mechanism for tailoring performance to specific mission requirements. Lower pitch settings favour climb by increasing low-speed thrust, while higher pitch settings improve cruise performance by better matching higher advance ratios. Engine scaling highlights the transition between power-limited and drag-limited regimes. While increasing engine power significantly improves climb performance, its effect on maximum speed becomes limited once drag dominates. Overall, the optimal propulsion configuration represents a balance between aerodynamic efficiency, engine capability, and mission objectives, rather than maximising a single performance metric.

9 Conclusion

This study analysed the performance of an APC20x8 propeller using blade element momentum theory and matched it with a given engine and airframe.

The results show that propeller performance is strongly influenced by advance ratio and RPM, with efficiency peaking at intermediate operating conditions. Engine-propeller matching demonstrated that maximum flight speed is determined by the balance between thrust and drag, while climb performance depends on excess thrust.

Variations in propeller pitch and engine size highlight important trade-offs in propulsion system design. Increasing pitch improves maximum speed but reduces climb performance, while increasing engine power significantly enhances climb capability with limited impact on maximum speed.

Overall, the analysis provides valuable insight into the interaction between propeller, engine, and airframe, and highlights the importance of selecting appropriate design parameters for optimal aircraft performance.

Bibliography

A Appendix

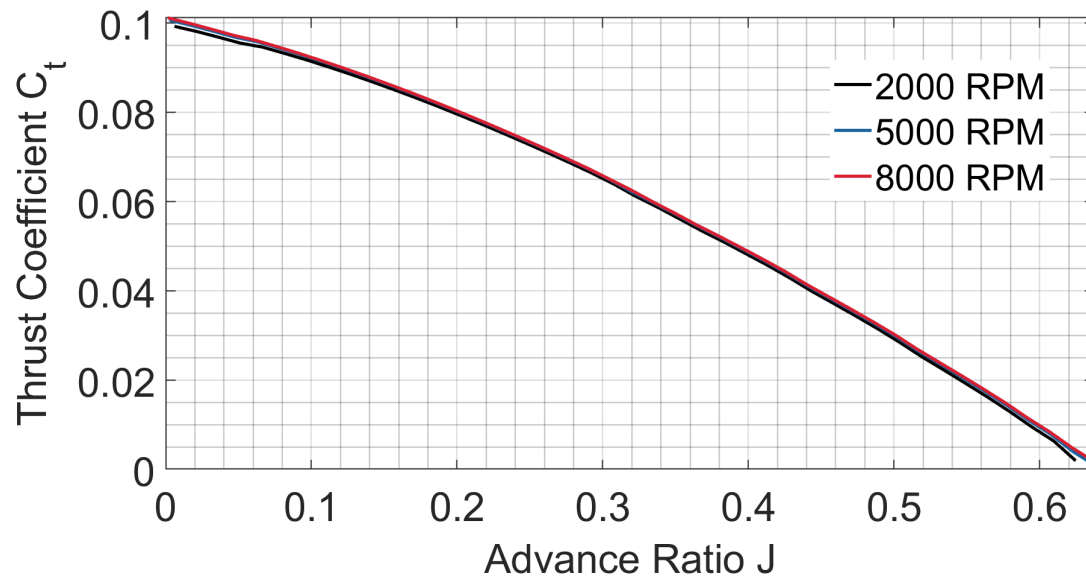


Figure 9: C_t vs advance ratio J

B Main MATLAB Code

Listing 1: Main code for propellor geometry and xfoil data

```

1 clear; close all; clc
2
3 %% ---- Propeller Geometry ----
4 % Select propeller (update if needed for your case)
5 PropellerName = 'APC20x8';
6
7 % Define radial positions from 0.1R to tip (10 elements as
  required)
8 r1 = linspace(0.1, 1, 10);
9
10 % Get chord and pitch distributions from provided function
11 [chord, pitch] = PropellerDimension(r1, PropellerName);
12
13 %% ---- Colour setup (just for nicer plots) ----
14 myred      = [216 30 49]/255;
15 myblue     = [27 99 157]/255;
16 myblack    = [0 0 0]/255;
17 mygreen    = [0 128 0]/255;
18 mycyan     = [2 169 226]/255;
19 myyellow   = [251 194 13]/255;
20 mygray     = [89 89 89]/255;
21
22 % Set default plotting colour order
23 set(groot, 'defaultAxesColorOrder', [myblack; myblue; myred; mygreen;
  myyellow; mycyan; mygray]);

```



```

24
25 %% ---- General plot formatting ----
26 alw = 1;           % axis line width
27 fsz = 22;          % font size (kept large for report figures)
28 lw = 2.5;          % line thickness
29 msz = 8;           % marker size
30
31 set(0, 'defaultLineLineWidth', lw);
32 set(0, 'defaultLineMarkerSize', msz);
33
34 %% ---- Figure 1: Pitch Distribution ----
35 hFig = figure(1); clf
36 set(gcf, 'Color', [1 1 1])
37 set(gca, 'Color', [1 1 1])
38 hold on
39 grid on
40 grid minor
41
42 % Convert pitch from radians to degrees for plotting
43 plot(r1, pitch * 180/pi)
44
45 xlabel('Normalised radius r/R')
46 ylabel('Pitch angle (deg)')
47 xlim([0.1 1])
48 box on
49
50 % Make the plot look clean and consistent
51 set(gca, 'LineWidth', alw)
52 set(gca, 'GridLineStyle', '-')
53 set(gca, 'MinorGridLineStyle', '-')
54 set(gca, 'GridColor', 'k')
55 set(gca, 'MinorGridColor', 'k')
56 set(findall(hFig, '-property', 'FontSize'), 'FontSize', fsz)
57
58 % Export high-quality image for report
59 print -dpng -r300 Pitch_Distribution
60
61 %% ---- Figure 2: Chord Distribution ----
62 hFig = figure(2); clf
63 set(gcf, 'Color', [1 1 1])
64 set(gca, 'Color', [1 1 1])
65 hold on
66 grid on
67 grid minor
68
69 plot(r1, chord)
70
71 xlabel('Normalised radius r/R')
72 ylabel('Chord (m)')
73 xlim([0.1 1])
74 box on

```

```

75
76 set(gca,'LineWidth',alw)
77 set(gca,'GridLineStyle','--')
78 set(gca,'MinorGridLineStyle','--')
79 set(gca,'GridColor','k')
80 set(gca,'MinorGridColor','k')
81 set(findall(hFig,'-property','FontSize'),'FontSize',fsz)
82
83 print -dpng -r300 Chord_Distribution
84
85 %% ---- Read XFoil Data ----
86 % Load airfoil data for different Reynolds numbers
87 Re500 = readmatrix('Naca4412Re500');
88 Re650 = readmatrix('Naca4412Re650');
89 Re800 = readmatrix('Naca4412Re800');
90 Re1000 = readmatrix('Naca4412Re1000');
91
92 % Extract columns (alpha, CL, CD)
93 alpha500 = Re500(:,1); c1500 = Re500(:,2); cd500 = Re500
    (:,3);
94 alpha650 = Re650(:,1); c1650 = Re650(:,2); cd650 = Re650
    (:,3);
95 alpha800 = Re800(:,1); c1800 = Re800(:,2); cd800 = Re800
    (:,3);
96 alpha1000 = Re1000(:,1); c11000 = Re1000(:,2); cd1000 = Re1000
    (:,3);
97
98 %% ---- Figure 3: CL vs Alpha ----
99 hFig = figure(3); clf
100 set(gcf,'Color',[1 1 1])
101 set(gca,'Color',[1 1 1])
102 hold on
103 grid on
104 grid minor
105
106 % Plot lift curves (higher Re should generally give better
    performance)
107 plot(alpha650, c1650)
108 plot(alpha800, c1800)
109 plot(alpha1000, c11000)
110
111 xlabel('Angle of attack \alpha (deg)')
112 ylabel('Lift coefficient C_L')
113 xlim([-15 15])
114 box on
115
116 set(gca,'LineWidth',alw)
117 set(gca,'GridLineStyle','--')
118 set(gca,'MinorGridLineStyle','--')
119 set(gca,'GridColor','k')
120 set(gca,'MinorGridColor','k')

```

```

121 set(findall(hFig, '-property', 'FontSize'), 'FontSize', fsz)
122
123 hleg = legend('Re = 650k', 'Re = 800k', 'Re = 1,000k');
124 set(hleg, 'EdgeColor', hleg.Color)
125 set(hleg, 'Location', 'northwest')
126
127 print -dpng -r300 Cl_vs_alpha
128
129 %% ---- Figure 4: CD vs Alpha ----
130 hFig = figure(4); clf
131 set(gcf, 'Color', [1 1 1])
132 set(gca, 'Color', [1 1 1])
133 hold on
134 grid on
135 grid minor
136
137 % Plot drag curves (expect lower CD at higher Re, especially pre-
    stall)
138 plot(alpha650, cd650)
139 plot(alpha800, cd800)
140 plot(alpha1000, cd1000)
141
142 xlabel('Angle of attack \alpha (deg)')
143 ylabel('Drag coefficient C_D')
144 ylim([0 0.06]) % limit to relevant range for clarity
145 xlim([-15 15])
146 box on
147
148 set(gca, 'LineWidth', alw)
149 set(gca, 'GridLineStyle', '-')
150 set(gca, 'MinorGridLineStyle', '-')
151 set(gca, 'GridColor', 'k')
152 set(gca, 'MinorGridColor', 'k')
153 set(findall(hFig, '-property', 'FontSize'), 'FontSize', fsz)
154
155 hleg = legend('Re = 650k', 'Re = 800k', 'Re = 1,000k');
156 set(hleg, 'EdgeColor', hleg.Color)
157 set(hleg, 'Location', 'north')
158
159 print -dpng -r300 Cd_vs_alpha
160
161 %% ---- Reset defaults ----
162 % Clean up so future plots are not affected
163 set(groot, 'defaultAxesColorOrder', 'remove')

```

Listing 2: Blade element code

```

1 %% Comments
2 % This m-file is based on the file created by Doug Auld for the
3 % Aerodynamics for students.
4 % Additional comments have been added by Dries Verstraete to
    assist students in

```

```

5 % AER03261/AER09261 with their propeller assignment
6
7 clear;
8 close all
9 clc
10
11 filenameprop          = 'testprop.mat';
12
13 %% Figure formatting
14 myred                  = [216 30 49]/255;
15 myblue                 = [27 99 157]/255;
16 myblack                = [0 0 0]/255;
17 mygreen                = [0 128 0]/255;
18 mycyan                 = [2 169 226]/255;
19 myyellow               = [251 194 13]/255;
20 mygray                 = [89 89 89]/255;
21 set(groot,'defaultAxesColorOrder',[myblack;myblue;myred;mygreen;
    myyellow;mycyan;mygray]);
22 alw                    = 1;
23 fsz                    = 26;
24 lw                     = 2.5;
25 msz                    = 40;
26 set(0,'defaultLineLineWidth',lw);
27 set(0,'defaultLineMarkerSize',msz);
28
29 %% Propeller Geometry
30 NrOfElements           = 10;
31 chordvec               =
    [0.027977708561170,0.034299337193030,0.040541124695084,0.042004958933961,
32 pitchvec               =
    [0.592297851384980,0.586232564288562,0.453445862572999,0.328392057826073,
33
34 dia                    = 0.5;
35 R                      = dia/2.0;
36
37 %% Blade segments
38 xs                     = 0.1*R;
39 xt                     = R;
40 rstep                  = (xt-xs)/(NrOfElements-1);
41 r1                     = (xs:rstep:xt);
42
43 %% Atmospheric conditions
44 p                      = 82750;
45 T                      = 277.23;
46 R                      = 287.05;
47 rho                   = p/R/T;
48 mu                     = 1.458e-6*T^1.5/(T+110.4);
49
50 %% Flight speed settings

```

```

51 Vmin                = 0.1;
52 NrOfVs              = 100;
53
54 %% Aerodynamic data
55 filenames.Re50       = 'Naca4412Re50';
56 filenames.Re100      = 'Naca4412Re100';
57 filenames.Re200      = 'Naca4412Re200';
58 filenames.Re350      = 'Naca4412Re350';
59 filenames.Re500      = 'Naca4412Re500';
60 filenames.Re650      = 'Naca4412Re650';
61 filenames.Re800      = 'Naca4412Re800';
62 filenames.Re1000     = 'Naca4412Re1000';
63
64 astall = -8.9; % in DEGREES
65
66 [alfa500,cl500,cd500,Remat,Cd0mat] = dragdata(filenames,astall);
67
68 %% Convergence criterion
69 accuracy                = 1e-3;
70
71 %% RPM Loop
72 RPM_list = [2000, 5000, 8000];
73
74 Results(length(RPM_list)) = struct(...
75     'RPM',[],'Vvec',[],'J',[],'t',[],'q',[],'eff',[],'T',[],'Q'
76     ,[],'P',[]);
77
78 for rpmIdx = 1:length(RPM_list)
79     RPM    = RPM_list(rpmIdx);
80     n      = RPM / 60;
81     omega  = n * 2 * pi;
82     Vmax   = n * dia * 1.5;
83     Vvec   = linspace(Vmin, Vmax, NrOfVs);
84
85     t      = NaN(1,length(Vvec));
86     q      = NaN(1,length(Vvec));
87     J      = NaN(1,length(Vvec));
88     eff    = NaN(1,length(Vvec));
89     ickchk = NaN(1,length(Vvec));
90     T      = NaN(1,length(Vvec));
91     Q      = NaN(1,length(Vvec));
92     P      = NaN(1,length(Vvec));
93
94     for cntrV = 1:length(Vvec)
95         V      = Vvec(cntrV);
96         thrust = 0.0;
97         torque  = 0.0;
98
99         for j = 1:size(r1,2)
100             rad      = r1(j);

```

```

101     pitch      = pitchvec(j);
102     chord      = chordvec(j);
103     theta      = pitch;
104     a          = 0.1;
105     b          = 0.01;
106     finished   = false;
107     sum        = 1;
108     itercheck  = 0;
109
110     while (~finished)
111         V0      = V*(1+a);
112         V2      = omega*rad*(1-b);
113         phi     = atan2(V0,V2);
114         alpha   = theta-phi;
115         Vlocal  = sqrt(V0*V0+V2*V2);
116         Re      = rho*chord*Vlocal/mu;
117
118         cl = interp1(alfa500,c1500,alpha,'spline','extrap
119             ');
120         cd = interp1(alfa500,cd500,alpha,'spline','extrap
121             ');
122         if alpha < alfa500(1)
123             cl = c1500(1);
124             cd = cd500(1) + 0.1;
125         elseif alpha > alfa500(end)
126             cl = c1500(end);
127             cd = cd500(end) + 0.1;
128         end
129         factor = interp1(Remat,Cd0mat,Re/500000,'spline',
130             'extrap');
131         cd      = cd*factor;
132
133         DtDr = 0.5*rho*Vlocal*Vlocal*2.0*chord*(cl*cos(
134             phi)-cd*sin(phi));
135         DqDr = 0.5*rho*Vlocal*Vlocal*2.0*chord*rad*(cd*
136             cos(phi)+cl*sin(phi));
137
138         tem1 = DtDr/(4.0*pi*rad*rho*V*V*(1+a));
139         tem2 = DqDr/(4.0*pi*rad*rad*rad*rho*V*(1+a)*omega
140             );
141         anew = 0.5*(a+tem1);
142         bnew = 0.5*(b+tem2);
143
144         if (abs((anew-a)/a)<accuracy)
145             if (abs((bnew-b)/b)<accuracy)
146                 finished = true;
147             end
148         end
149         a = anew;
150         b = bnew;
151         sum = sum+1;

```

```

146         if (sum>500)
147             finished = true;
148             itercheck = 1;
149         end
150     end
151
152     if (isinf(a)||isinf(b)||isnan(a)||isnan(b))
153         thrust = thrust;
154         torque = torque;
155     else
156         thrust = thrust + DtDr*rrstep;
157         torque = torque + DqDr*rrstep;
158     end
159 end
160
161 t(cntrV)      = thrust/(rho*n*n*dia*dia*dia*dia);
162 q(cntrV)      = torque/(rho*n*n*dia*dia*dia*dia*dia);
163 J(cntrV)      = V/(n*dia);
164 eff(cntrV)    = J(cntrV)/2.0/pi*t(cntrV)/q(cntrV);
165 icheck(cntrV) = itercheck;
166 T(cntrV)      = thrust;
167 Q(cntrV)      = torque;
168 P(cntrV)      = 2*pi*n*torque;
169 end
170
171 ind1          = find(t<0);
172 ind2          = find(q<0);
173 ind           = union(ind1,ind2);
174 t(ind)        = []; q(ind)      = []; J(ind)      = [];
175 eff(ind)      = []; icheck(ind)= []; T(ind)      = [];
176 Q(ind)        = []; P(ind)      = []; Vvec(ind) = [];
177
178 Results(rpmIdx).RPM = RPM;
179 Results(rpmIdx).Vvec = Vvec;
180 Results(rpmIdx).J    = J;
181 Results(rpmIdx).t    = t;
182 Results(rpmIdx).q    = q;
183 Results(rpmIdx).eff  = eff;
184 Results(rpmIdx).T    = T;
185 Results(rpmIdx).Q    = Q;
186 Results(rpmIdx).P    = P;
187
188 end
189
190 %% Plotting
191 colors      = [myblack; myblue; myred];
192 legendLabels = {'2000 RPM', '5000 RPM', '8000 RPM'};
193
194 %% Figure 1 - Thrust & Power vs Flight Speed
195 hFig = figure(1); clf
196 set(gcf, 'Color', [1 1 1])

```

```

197 set(gca,'Color',[1 1 1])
198 hold on
199 grid on
200 grid minor
201
202 yyaxis left
203 for i = 1:3
204     plot(Results(i).Vvec, Results(i).T, '--', 'Color', colors(i
205         ,:));
206 end
207 ylabel('Thrust (N)')
208 yyaxis right
209 for i = 1:3
210     plot(Results(i).Vvec, Results(i).P, '-', 'Color', colors(i,:
211         ));
212 end
213 ylabel('Power (W)')
214 xlabel('Flight Speed V (m/s)')
215
216 box on
217 set(gca,'LineWidth',alw)
218 set(gca,'GridLineStyle','--')
219 set(gca,'MinorGridLineStyle','--')
220 set(gca,'GridColor','k')
221 set(gca,'MinorGridColor','k')
222 set(findall(hFig,'-property','FontSize'),'FontSize',fsz)
223
224 hleg = legend([strcat(legendLabels,' - T'), strcat(legendLabels,'
225     - P')]);
226 set(hleg,'EdgeColor',hleg.Color)
227 set(hleg,'Location','best')
228
229 print -dpng -r300 Thrust_Power_vs_Speed
230
231 %% Figure 2 - Thrust & Power vs Advance Ratio
232 hFig = figure(2); clf
233 set(gcf,'Color',[1 1 1])
234 set(gca,'Color',[1 1 1])
235 hold on
236 grid on
237 grid minor
238
239 yyaxis left
240 for i = 1:3
241     plot(Results(i).J, Results(i).T, '--', 'Color', colors(i,:));
242 end
243 ylabel('Thrust (N)')
244 yyaxis right
245 for i = 1:3
246     plot(Results(i).J, Results(i).P, '-', 'Color', colors(i,:));
247 end

```



```

245 ylabel('Power (W)')
246 xlabel('Advance Ratio J')
247
248 box on
249 set(gca,'LineWidth',alw)
250 set(gca,'GridLineStyle','--')
251 set(gca,'MinorGridLineStyle','--')
252 set(gca,'GridColor','k')
253 set(gca,'MinorGridColor','k')
254 set(findall(hFig,'-property','FontSize'),'FontSize',fsz)
255
256 hleg = legend([strcat(legendLabels,' - T'), strcat(legendLabels,'
    - P')]);
257 set(hleg,'EdgeColor',hleg.Color)
258 set(hleg,'Location','best')
259
260 print -dpng -r300 Thrust_Power_vs_J
261
262 %% Figure 3 - Thrust Coefficient vs Advance Ratio
263 hFig = figure(3); clf
264 set(gcf,'Color',[1 1 1])
265 set(gca,'Color',[1 1 1])
266 hold on
267 grid on
268 grid minor
269
270 for i = 1:3
271     plot(Results(i).J, Results(i).t, 'Color', colors(i,:));
272 end
273 xlabel('Advance Ratio J')
274 ylabel('Thrust Coefficient C_t')
275
276 box on
277 set(gca,'LineWidth',alw)
278 set(gca,'GridLineStyle','--')
279 set(gca,'MinorGridLineStyle','--')
280 set(gca,'GridColor','k')
281 set(gca,'MinorGridColor','k')
282 set(findall(hFig,'-property','FontSize'),'FontSize',fsz)
283
284 hleg = legend(legendLabels);
285 set(hleg,'EdgeColor',hleg.Color)
286 set(hleg,'Location','best')
287
288 print -dpng -r300 Ct_vs_J
289
290 %% Figure 4 - Torque Coefficient vs Advance Ratio
291 hFig = figure(4); clf
292 set(gcf,'Color',[1 1 1])
293 set(gca,'Color',[1 1 1])
294 hold on

```

```
295 grid on
296 grid minor
297
298 for i = 1:3
299     plot(Results(i).J, Results(i).q, 'Color', colors(i,:));
300 end
301 xlabel('Advance Ratio J')
302 ylabel('Torque Coefficient Cq')
303
304 box on
305 set(gca,'LineWidth',alw)
306 set(gca,'GridLineStyle','--')
307 set(gca,'MinorGridLineStyle','--')
308 set(gca,'GridColor','k')
309 set(gca,'MinorGridColor','k')
310 set(findall(hFig,'-property','FontSize'),'FontSize',fsz)
311
312 hleg = legend(legendLabels);
313 set(hleg,'EdgeColor',hleg.Color)
314 set(hleg,'Location','best')
315
316 print -dpng -r300 Cq_vs_J
317
318 %% Figure 5 - Efficiency vs Advance Ratio
319 hFig = figure(5); clf
320 set(gcf,'Color',[1 1 1])
321 set(gca,'Color',[1 1 1])
322 hold on
323 grid on
324 grid minor
325
326 for i = 1:3
327     plot(Results(i).J, Results(i).eff, 'Color', colors(i,:));
328 end
329 xlabel('Advance Ratio J')
330 ylabel('Propulsive Efficiency \eta')
331
332 box on
333 set(gca,'LineWidth',alw)
334 set(gca,'GridLineStyle','--')
335 set(gca,'MinorGridLineStyle','--')
336 set(gca,'GridColor','k')
337 set(gca,'MinorGridColor','k')
338 set(findall(hFig,'-property','FontSize'),'FontSize',fsz)
339
340 hleg = legend(legendLabels);
341 set(hleg,'EdgeColor',hleg.Color)
342 set(hleg,'Location','best')
343
344 print -dpng -r300 Efficiency_vs_J
345
```

```

346 %% Reset default colour order
347 set(groot,'defaultAxesColorOrder','remove')
348
349 %% Save propeller data for engine matching
350 for i = 1:3
351     Propeller.AdvanceRatio = Results(i).J;
352     Propeller.Efficiency   = Results(i).eff;
353     Propeller.ThrustCoeff  = Results(i).t;
354     Propeller.TorqueCoeff  = Results(i).q;
355     Propeller.Diameter     = dia;
356     Propeller.RotSpeed     = Results(i).RPM / 60;
357     fname = sprintf('testprop_%dRPM.mat', Results(i).RPM);
358     save(fname, 'Propeller');
359 end

```

Listing 3: Engine matching code

```

1 %% Assignment 1
2 clear
3 close all
4 clc
5
6 %% ---- Flight conditions ----
7 % Set atmospheric conditions (given / assumed)
8 FlightCondition.Pressure      = 82750;      % Pa
9 FlightCondition.Temperature   = 277.23;     % K
10 FlightCondition.R             = 287.05;     % gas constant
11 FlightCondition.Density       = FlightCondition.Pressure / ...
12                               FlightCondition.R /
13                               FlightCondition.Temperature;
14 FlightCondition.g             = 9.80665;    % gravity
15 FlightCondition.RoC           = 0;          % zero for cruise
16 case
17
18 %% ---- Speed range ----
19 % Define flight speed sweep for performance analysis
20 FlightSpeed.Vmin             = 5;
21 FlightSpeed.Vmax             = 50;
22 FlightSpeed.NrOfPoints       = 100;
23 FlightCondition.Speed        = linspace(FlightSpeed.Vmin,
24    FlightSpeed.Vmax,FlightSpeed.NrOfPoints);
25
26 % Basic aero quantities
27 FlightCondition.ClimbAngle    = atan(FlightCondition.RoC ./
28    FlightCondition.Speed);
29 FlightCondition.DynPressure   = 0.5 * FlightCondition.Density
30    .* FlightCondition.Speed.^2;
31
32 %% ---- Aircraft properties ----
33 Aircraft.m                   = 13.5;        % mass
34 Aircraft.Sw                  = 0.55;        % wing area
35 Aircraft.b                   = 2.9;         % wingspan

```

```

31 Aircraft.CD0          = 0.03;          % parasitic drag
32 Aircraft.e            = 0.90;          % Oswald efficiency
33 Aircraft.CL0          = 0.28;          % lift offset
34
35 Aircraft.W            = Aircraft.m * FlightCondition.g;
36 Aircraft.AR           = Aircraft.b^2 / Aircraft.Sw;
37
38 % Lift coefficient required for steady flight
39 FlightCondition.CL     = Aircraft.W ./ ...
40   (FlightCondition.DynPressure .* Aircraft.Sw .* cos(
41     FlightCondition.ClimbAngle));
42
43 % Drag model (parabolic)
44 FlightCondition.CD     = Aircraft.CD0 + ...
45   (FlightCondition.CL - Aircraft.CL0).^2 ./ (pi * Aircraft.e *
46     Aircraft.AR);
47
48 % Drag force
49 FlightCondition.Drag   = FlightCondition.DynPressure .*
50   Aircraft.Sw .* FlightCondition.CD;
51
52 % Thrust required (includes climb term, currently zero here)
53 FlightCondition.ThrustRequired = FlightCondition.Drag + ...
54   Aircraft.W * sin(FlightCondition.ClimbAngle);
55
56 %% ---- Climb case ----
57 % Introduce a realistic climb rate to compare against cruise
58 RoC_climb = 2.5;
59
60 ClimbAngle_climb = atan(RoC_climb ./ FlightCondition.Speed);
61
62 % Extra thrust needed due to climb
63 ThrustRequired_climb = FlightCondition.Drag + ...
64   Aircraft.W .* sin(ClimbAngle_climb);
65
66 %% ---- Quick sanity check ----
67 figure
68 plot(FlightCondition.Speed, FlightCondition.ThrustRequired, '
69   LineWidth', 2)
70 xlabel('Flight Speed (m/s)')
71 ylabel('Required Thrust (N)')
72 title('Required Thrust vs Flight Speed')
73 grid on
74 box on
75
76 %% ---- Propeller data ----
77 % Load precomputed propeller performance map
78 filenameprop = 'testprop_8000RPM.mat';
79 load(filenameprop)
80
81 n          = Propeller.RotSpeed;

```

```

78 dia      = Propeller.Diameter;
79 J        = Propeller.AdvanceRatio;
80 eff      = Propeller.Efficiency;
81 t        = Propeller.ThrustCoeff;
82 q        = Propeller.TorqueCoeff;
83 rho      = FlightCondition.Density;
84
85 % Remove invalid entries (important for interpolation)
86 valid    = ~(isnan(J) | isnan(t) | isnan(q) | isnan(eff));
87 J        = J(valid);
88 t        = t(valid);
89 q        = q(valid);
90 eff      = eff(valid);
91
92 %% ---- Engine data ----
93 % Get engine performance map
94 [EngineMap] = EngineSpecs(FlightCondition);
95
96 ThrottleVec = [40 60 80 100];
97 MatchAll    = struct();
98
99 %% ---- Engine prop matching ----
100 % Loop through throttle settings
101 for iT = 1:length(ThrottleVec)
102
103     Throttle = ThrottleVec(iT);
104
105     % Select appropriate engine row (scaled by throttle)
106     RPMrow    = round(length(EngineMap.RPM(:,1))/100 *
107         Throttle);
108     EngineRPM    = EngineMap.RPM(RPMrow,:);
109     EnginePower  = EngineMap.Power(RPMrow,:);
110     EngineTorque  = EngineMap.Torque(RPMrow,:);
111     EngineFuelFlow = EngineMap.FuelFlow(RPMrow,:);
112     nvec        = EngineRPM / 60;
113
114     % Preallocate
115     Match.J      = NaN(size(FlightCondition.Speed));
116     Match.RPM    = NaN(size(FlightCondition.Speed));
117     Match.EngineTorque = NaN(size(FlightCondition.Speed));
118     Match.PropTorque = NaN(size(FlightCondition.Speed));
119     Match.PropThrust = NaN(size(FlightCondition.Speed));
120     Match.PropEff  = NaN(size(FlightCondition.Speed));
121     Match.EngineFuelFlow = NaN(size(FlightCondition.Speed));
122     Match.EnginePower = NaN(size(FlightCondition.Speed));
123     Match.PropPower = NaN(size(FlightCondition.Speed));
124     indQmatch    = NaN(size(FlightCondition.Speed));
125
126     for cntrV = 1:length(FlightCondition.Speed)
127
128         Speed = FlightCondition.Speed(cntrV);

```

```

128
129 % Convert speed to advance ratio
130 Jvec = Speed ./ (nvec * dia);
131
132 % Interpolate prop performance at this J
133 qvec = interp1(J,q,Jvec,'spline',NaN);
134 tvec = interp1(J,t,Jvec,'spline',NaN);
135 effvec = interp1(J,eff,Jvec,'spline',NaN);
136
137 % Convert coefficients to physical values
138 PropTorqueVec = qvec .* rho .* nvec.^2 .* dia.^5;
139 PropThrustVec = tvec .* rho .* nvec.^2 .* dia.^4;
140 PropPowerVec = 2*pi*nvec .* PropTorqueVec;
141
142 % Find operating point where prop torque matches engine
    torque
143 ind = find(PropTorqueVec >= EngineTorque,1,'first');
144
145 if ~isempty(ind)
146     indQmatch(cntrV) = ind;
147     Match.J(cntrV) = Jvec(ind);
148     Match.RPM(cntrV) = EngineRPM(ind);
149     Match.EngineTorque(cntrV) = EngineTorque(ind);
150     Match.PropTorque(cntrV) = PropTorqueVec(ind);
151     Match.PropThrust(cntrV) = PropThrustVec(ind);
152     Match.PropEff(cntrV) = effvec(ind);
153     Match.EngineFuelFlow(cntrV) = EngineFuelFlow(ind);
154     Match.EnginePower(cntrV) = EnginePower(ind);
155     Match.PropPower(cntrV) = PropPowerVec(ind);
156 end
157 end
158
159 % Remove invalid matches (edge artefacts)
160 indFalse = find(indQmatch == 1);
161 Match.J(indFalse) = NaN;
162 Match.RPM(indFalse) = NaN;
163 Match.EngineTorque(indFalse) = NaN;
164 Match.PropTorque(indFalse) = NaN;
165 Match.PropThrust(indFalse) = NaN;
166 Match.PropEff(indFalse) = NaN;
167 Match.EngineFuelFlow(indFalse) = NaN;
168 Match.EnginePower(indFalse) = NaN;
169 Match.PropPower(indFalse) = NaN;
170
171 MatchAll(iT).Throttle = Throttle;
172 MatchAll(iT).Match = Match;
173 end
174
175 %% ---- Performance extraction ----
176 % Extract key performance metrics (cruise + climb)
177 nT = length(ThrottleVec);

```

```

178
179 Vmax_all      = NaN(nT,1);
180 RoCmax_all    = NaN(nT,1);
181 Vbest_all     = NaN(nT,1);
182
183 for iT = 1:nT
184
185     Match = MatchAll(iT).Match;
186
187     % Cruise condition (where thrust balances drag)
188     diffThrust = Match.PropThrust - FlightCondition.
        ThrustRequired;
189     indCruise  = find(diffThrust >= 0,1,'last');
190
191     if ~isempty(indCruise)
192         Vmax_all(iT) = FlightCondition.Speed(indCruise);
193     end
194
195     % Rate of climb calculation
196     RoCvec = (Match.PropThrust - FlightCondition.Drag) .* ...
        FlightCondition.Speed / Aircraft.W;
197
198     [RoCmax_all(iT), indRoC] = max(RoCvec);
199
200     Vbest_all(iT) = FlightCondition.Speed(indRoC);
201 end
202
203 %% ---- Plot: thrust vs speed ----
204 % Shows where T = D and climb margin
205 for iT = 1:length(ThrottleVec)
206     plot(FlightCondition.Speed, MatchAll(iT).Match.PropThrust)
207 end
208 plot(FlightCondition.Speed, FlightCondition.Drag, '--')
209 plot(FlightCondition.Speed, ThrustRequired_climb, '--')
210
211 xlabel('Flight Speed (m/s)')
212 ylabel('Thrust (N)')
213

```